📄 جزوه سیستم عامل - فصل ۶: ابزار همگامی (Synchronization Tools)

(بخش اول: مفاهیم پایه، شرط مسابقه و راهکارهای نرم‌افزاری)

۱. مقدمه: چرا به همگامی نیاز داریم؟

در سیستم‌های امروزی، پردازه‌ها (Process) می‌توانند **همکار (Cooperating)** باشند.

- **تعریف پردازه همکار:** پردازه‌ای است که می‌تواند روی اجرای سایر پردازه‌ها اثر بگذارد یا از آن‌ها اثر بپذیرد.
- **روش اشتراک‌گذاری:** این پردازه‌ها از طریق «فضای آدرس مشترک» (کد و داده) یا «اشتراک فایل/پیغام» با هم در ارتباط هستند.
- **مشکل اصلی:** دسترسی هم‌زمان (Concurrent) به داده‌های مشترک می‌تواند باعث **ناسازگاری داده‌ها (Data Inconsistency)** شود.
- **هدف:** ما نیاز به مکانیزم‌هایی داریم که اجرای مرتب پردازه‌ها را تضمین کند تا داده‌ها صحیح باقی بمانند.

۲. شرط مسابقه (Race Condition)

مهم‌ترین مفهوم این فصل است که پایه‌ی تمام مباحث بعدی را می‌سازد.

> **تعریف:** وضعیتی که در آن چندین پردازه به طور هم‌زمان داده‌های یکسانی را دستکاری می‌کنند و نتیجه نهایی اجرا، بستگی به این دارد که کدام پردازه زودتر کارش را تمام کند (به ترتیب اجرا وابسته است).

مثال تشریحی: مسئله تولیدکننده - مصرف‌کننده

فرض کنید متغیری به نام count داریم که تعداد آیتم‌های موجود در بافر را نگه می‌دارد.

- **تولیدکننده (Producer):** یک آیتم اضافه می‌کند -> ++count
- **مصرف‌کننده (Consumer):** یک آیتم برمی‌دارد -> –count

اگرچه دستور count++ در زبان برنامه‌نویسی یک خط است، اما در زبان ماشین (سطح پایین) شامل **۳ مرحله** است:

1. انتقال مقدار متغیر به ثبات (register1 = count)
2. افزایش مقدار ثبات (register1 = register1 + 1)
3. ذخیره ثبات در حافظه (count = register1)

سناریوی بروز خطا (بسیار مهم برای امتحان):

فرض کنید مقدار اولیه count = 5 است. اگر تولیدکننده و مصرف‌کننده هم‌زمان اجرا شوند، ممکن است چنین تداخلی رخ دهد:

1. **تولیدکننده:** مقدار ۵ را در register1 می‌خواند.
2. **تولیدکننده:** register1 را ۶ می‌کند. (هنوز در حافظه اصلی ننوشته است).
3. *(در اینجا CPU از تولیدکننده گرفته شده و به مصرف‌کننده داده می‌شود).*
4. **مصرف‌کننده:** مقدار ۵ (که هنوز در حافظه تغییر نکرده) را در register2 می‌خواند.
5. **مصرف‌کننده:** register2 را ۴ می‌کند.
6. **مصرف‌کننده:** مقدار ۴ را در count می‌نویسد (الان count=4).
7. *(CPU به تولیدکننده برمی‌گردد).*
8. **تولیدکننده:** مقدار ۶ (که در ثباتش بود) را در count می‌نویسد (الان count=6).

نتیجه: مقدار واقعی باید ۵ می‌ماند، اما به اشتباه ۶ شد (یا ممکن بود ۴ شود). به این وضعیت **Race Condition** می‌گویند.

۳. مسئله بخش بحرانی (Critical Section Problem)

برای حل مشکل بالا، باید پروتکلی طراحی کنیم که پردازه‌ها با هماهنگی از داده‌های مشترک استفاده کنند. هر پردازه یک قطعه کد دارد که به آن **بخش بحرانی** می‌گویند.

ساختار کلی کد یک پردازه:

1. **بخش ورود (Entry Section):** کدی که در آن پردازه "اجازه" ورود به ناحیه حساس را درخواست می‌کند.
2. **بخش بحرانی (Critical Section - CS):** بخشی از کد که در آن متغیرهای مشترک تغییر داده می‌شوند (مهم‌ترین بخش).

3. **بخش خروج (Exit Section):** کدی که پس از اتمام کار اجرا می‌شود تا اجازه را آزاد کند.
4. **بخش باقیمانده (Remainder Section):** سایر کدهای عادی برنامه.

💎 ۳ شرط الزامی برای حل مسئله (نکات طلایی)

هر راه حلی برای بخش بحرانی **باید** این سه ویژگی را داشته باشد:

انحصار متقابل (Mutual Exclusion):
- اگر پردازه P_i در بخش بحرانی خود باشد، **هیچ** پردازه دیگری حق ندارد در بخش بحرانی خود باشد.

پیشروی (Progress):
- اگر کسی در بخش بحرانی نیست و پردازه‌هایی می‌خواهند وارد شوند، فقط آن‌هایی که در "بخش باقیمانده" نیستند (یعنی کار دارند) باید تصمیم بگیرند چه کسی وارد شود. این تصمیم نباید تا ابد به تعویق بیفتد.

انتظار محدود (Bounded Waiting):
- باید سقفی (محدودیتی) برای تعداد دفعاتی که دیگران اجازه دارند وارد بخش بحرانی شوند (در حالی که یک پردازه منتظر است) وجود داشته باشد. (جلوگیری از گرسنگی پردازه).

۴. مدیریت همگامی در هسته سیستم‌عامل

خودِ سیستم‌عامل هم ممکن است دچار شرایط مسابقه شود (مثلاً هنگام تخصیص حافظه یا لیست پردازه‌ها). دو رویکرد برای طراحی هسته وجود دارد:

مزایا و معایب	عملکرد	نوع هسته
مزیت: ذاتاً عاری از شرط مسابقه است. **عیب:** برای سیستم‌های بلادرنگ مناسب نیست (کندی پاسخ).	وقتی پردازه‌ای در مد هسته اجرا می‌شود، تا خودش نخواهد خارج نمی‌شود (کسی نمی‌تواند قطعش کند).	**هسته غیرقابل پس‌گیر (Non-preemptive)**
مزیت: پاسخگویی سریع‌تر، مناسب برای Real-time. **عیب:** طراحی دشوار چون باید مراقب شرط مسابقه	اجازه می‌دهد پردازه‌ای که در مد هسته است، متوقف (Preempt) شود.	**هسته پس‌گیر (Preemptive)**

بود.

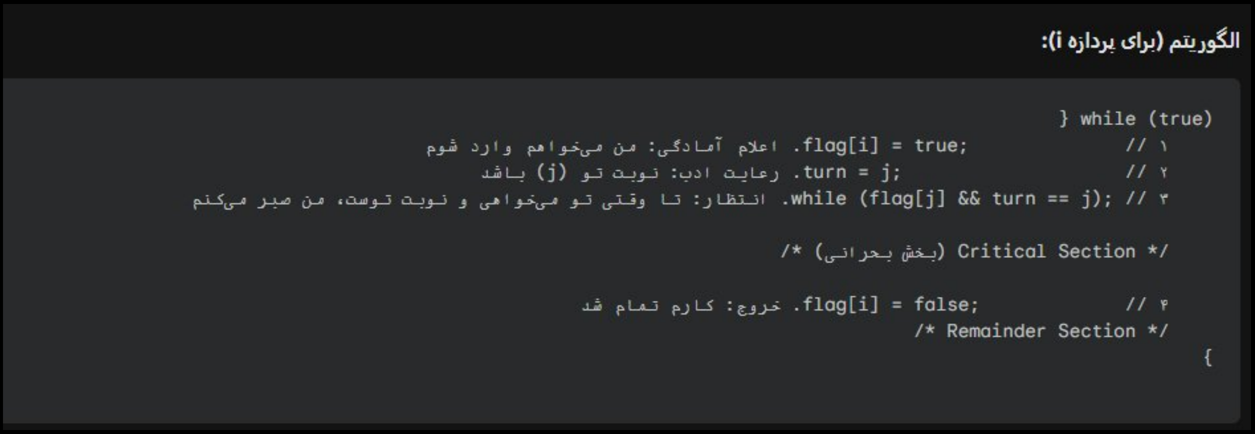
۵. راه حل پترسن (Peterson's Solution)

یک راه حل کلاسیک **نرم‌افزاری** برای مسئله بخش بحرانی که فقط برای **۲ پردازه** طراحی شده است.

متغیرهای مشترک:

1. int turn: نوبت کیست؟ (اگر turn == i باشد، پردازه i اجازه ورود دارد).
2. boolean flag[2]: علاقه به ورود. (اگر flag[i] == true یعنی پردازه i آماده ورود است).

الگوریتم (برای پردازه i):



الگوریتم (برای پردازه i):

```
                                          } while (true)
          اعلام آمادگی: من می‌خواهم وارد شوم .flag[i] = true;          // ۱
                 رعایت ادب: نوبت تو (j) باشد .turn = j;               // ۲
انتظار: تا وقتی تو می‌خواهی و نوبت توست، من صبر می‌کنم .while (flag[j] && turn == j); // ۳

                          /* (بخش بحرانی) Critical Section */

                    خروج: کارم تمام شد .flag[i] = false;              // ۴
                                   /* Remainder Section */
                                                            {
```

تحلیل راه حل پترسن:

1. **انحصار متقابل:** دارد. چون turn نمی‌تواند همزمان هم i باشد و هم j. پس یکی حتماً در حلقه while گیر می‌کند.
2. **پیشروی:** دارد. اگر کسی نخواهد وارد شود، flag او false است و دیگری وارد می‌شود.
3. **انتظار محدود:** دارد. هر کس حداکثر یک دور صبر می‌کند.

⚠ مشکل پترسن در کامپیوترهای مدرن

در معماری‌های جدید، پردازنده‌ها یا کامپایلرها برای سرعت بیشتر، ممکن است **ترتیب دستورات را تغییر دهند (Reordering)**.

- **مثال:** ممکن است دستور تغییر flag و turn جابجا اجرا شوند.

- **نتیجه:** راه حل پترسن روی ماشین‌های امروزی تضمین نمی‌کند درست کار کند و ممکن است هر دو پردازه وارد بخش بحرانی شوند.

۶. مقدمه‌ای بر پشتیبانی سخت‌افزاری

چون راهکارهای نرم‌افزاری (مثل پترسن) کند هستند یا روی سخت‌افزار مدرن تضمین شده نیستند، ما به کمک سخت‌افزار نیاز داریم.

سدهای حافظه (Memory Barriers)

- **مشکل:** سیستم ممکن است برای بهینه‌سازی، ترتیب اجرای دستورات را تغییر دهد.
- **راه حل:** دستوراتی سخت‌افزاری هستند که به پردازنده می‌گویند "صبر کن تا تمام تغییرات حافظه اعمال شوند" و جلوی تغییر ترتیب را می‌گیرند.
- **مدل‌های حافظه:**
    - **ترتیب قوی (Strongly ordered):** تغییرات یک پردازنده سریعاً توسط بقیه دیده می‌شود.
    - **ترتیب ضعیف (Weakly ordered):** تغییرات ممکن است بلافاصله دیده نشوند.

(پایان بخش ارسالی شما - صفحات ۱ تا ۱۰)